

1. Einleitung – Was ist das SD Graphic System?

Das SD Graphic System ist ein modulares Grafiksystem für kleine und kleinste Mikrocontroller.

Es trennt **Grafikdarstellung** und **Programmlogik**.

Die Darstellung erfolgt auf einem separaten Grafikcontroller, während der Host-Controller sich auf Sensorik, Steuerung und Kommunikation konzentrieren kann.

Der Grafikcontroller übernimmt:

- Darstellung von Screens
- Textausgabe
- Sprite-Animation
- Übergänge und Effekte
- SD-Karten-Streaming
- Tasteneingaben

Der Host sendet lediglich **kompakte Befehle (Tokens)** an den Grafikcontroller.

Dies reduziert:

- RAM-Verbrauch
- CPU-Last
- Entwicklungsaufwand für grafische Oberflächen.

|

2. Quick Start

Minimalbeispiel:

`#include <SGC.h>` oder `#include „SGC.h“` wenn die `SGC.h` im Projektverzeichnis liegt

```
void setup()
{
    sgc.begin();
    sgc.showScreen(0);
    sgc.setCursor(1,4);
    sgc.print("Hello World");
}

void loop()
{
}
```

Der Host sendet lediglich Befehle an den Grafikcontroller.

Die Darstellung erfolgt vollständig auf dem Grafikboard.

3. Screens

Ein Screen ist eine vollständige Displaydarstellung

Ein Screen enthält:

Displayauflösung / 8

Tiles.

Beispiel 128×64 Display:

1024 Bytes pro Screen

Screens liegen im RAW Format auf der SD-Karte in der Assetpipeline die die SD Oled Suite erstellt. Der Grafikcontroller kann dieses RAW Format direkt **sehr performant** streamen.

4. Animationen

Animationen bestehen aus aufeinanderfolgenden Screens.

Beispiel:

Frame 200

Frame 201

Frame 202

Frame 203

Der Client kann eine Animation starten mit:

PLAY 200 → 203

Die Animation läuft **autonom nebenläufig** auf dem Grafikcontroller. Bremst den Host nicht! → max 50 fps!

5. Sprites

Sprites sind bewegliche grafische Elemente.

Sprites bestehen aus:

Fonttiles

Ein Sprite kann mehrere Frames enthalten.

Beispiel:

sprite_from = 20

sprite_count = 4

Damit entsteht eine Animation aus n Tiles.

6. Patternbuffer

Der Grafikcontroller erzeugt optional einen Patternbuffer.

Dieser enthält:

1 Bit pro Tile

Der Patternbuffer kann verwendet werden für:

- Collision Detection
- Dirty Region Rendering
- Host-Analyse des Screens

7. Kommunikation

Der Host kommuniziert mit dem Grafikcontroller mit SoftwareSerial über ein Token-Protokoll.

Ein Befehl besteht aus:

Startbyte
Länge
Opcode
Parameter

Opcodes sind feste Byte Tokens.

8. Typischer Workflow

1. Grafik im Screeneditor erstellen
2. Assets exportieren
3. SD-Karte in Grafikcontroller einsetzen
4. Host sendet Befehle

Beispiel:

```
showScreen(24)  
print("Temperature")
```

9. Hardware

Das Grafikboard basiert auf:

- LGT8F328 Mikrocontroller
- OLED Display (Fast I²C)
- SD-Karte (SPI)
- optional Funkmodul (HC12)
- optionale Taster

10. Designphilosophie

Das System wurde entwickelt, um die grafische Darstellung vollständig vom Host-Controller zu trennen.

Vorteile:

- minimaler Ressourcenverbrauch auf dem Host
- sehr große Anzahl von Screens möglich → max. 32GB!
- einfache Erstellung komplexer Benutzeroberflächen
- schnelle Entwicklung durch Screeneditor-> Assetpipeline

Command Reference

Alle Befehle werden als **1-Byte Opcode** an den Grafikcontroller übertragen.

Ein Telegramm hat folgendes Format:

START | LEN | OPCODE | PARAMETER

Der Grafikcontroller interpretiert die Opcodes über einen internen Interpreter.

1. Screen Commands

Diese Befehle steuern die Darstellung kompletter Screens.

Command	Beschreibung
SHOW_SCREEN	Zeigt einen Screen aus der SD-Assetpipeline
SHOW_SCREENS	Spielt eine Animation von Screen A bis Screen B
SHOW_SCREENS_PART	Zeigt nur den oberen oder unteren Teil eines Screens
CLEAR_SCREEN	Löscht das Display
HOME	Cursor auf Position (0,0)
DRAW_SCREEN	Background/Foreground Zonenclipping
SET_SPAGE	Setzt die Screen-Page

2. Text Commands

Textausgabe erfolgt über Tiles oder Fonts.

Command	Beschreibung
PRINT	Text an aktueller Cursorposition ausgeben
PRINT_AT	Text an bestimmter Position
SET_CURSOR	Cursorposition setzen
CLEAR_LINE	Aktuelle Zeile löschen
CLEAR_LINES_UPPER	Zeilen oberhalb löschen
CLEAR_LINES_LOWER	Zeilen unterhalb löschen
PRINT_P	Proportionalschrift

3. Font / Tile Commands

Diese Befehle greifen direkt auf Font-Tiles zu.

Command	Beschreibung
DRAW_TILE	Einzelnes Tile ausFont zeichnen
DRAW_SD_TILE	Tile direkt von SD laden
SET_TPAGE	Tilepage wählen (z.B. normal / invertiert / proportional)
SET_WPAGE	Word-Page setzen (z.B. Sprache)
SD_STRING_X	Komplettes Wort horizontal ausgeben
SD_STRING_Y	Komplettes Wort vertikal ausgeben
SD_P_STRING	Proportionalschrift ausgeben
F_STRING	String mit FastFont aus Flash

4. Graphics Commands

Grafikprimitive.

Command	Beschreibung
SD_GFX_DRAW_LINE	Linie zeichnen
SD_GFX_DRAW_VERTIKAL_LINE	Vertikale Linie
SD_GFX_DRAW_RECTANGLE	Rechteck
SD_GFX_DRAW_FILLED_RECTANGLE	Gefülltes Rechteck
SD_GFX_DRAW_CIRCLE	Kreis

5. Sprite Commands

Sprites sind bewegliche Grafikelemente.

Command	Beschreibung
SET_SPRITE	Sprite konfigurieren
SET_SPRITE_POSITION	Spriteposition setzen
SPRITE	Sprite anzeigen

6. Patternbuffer Commands

Der Patternbuffer enthält eine Tile-Belegungsmap des aktuellen Screens.

Command	Beschreibung
SD_PATTERN_LOAD	Patternbuffer aus Screen erzeugen
SD_PATTERN_CLEAR	Patternbuffer löschen
SD_PATTERN_GET	Patternbuffer zum Host übertragen

Command	Beschreibung
SD_PATTERN_PUT	Patternbuffer setzen
SD_PATTERN_RECEIVE	Patterndaten empfangen
SD_COLLISION	Kollision prüfen

7. Display Settings

Displayparameter.

Command	Beschreibung
SET_FLIPMODE	Display drehen
SET_CONTRAST	Kontrast einstellen
SET_INVERSEFONT	Invertierten Font verwenden
SET_POWERSAVE	Display Power Save

8. IO Commands

Der Grafikcontroller kann auch einfache IO-Funktionen übernehmen.

Command	Beschreibung
SET_PINMODE	Pinmode setzen
SET_PIN	Digital pin setzen
SET_ANALOGWRITE	PWM ausgeben
GET_KEY	Tasteneingabe lesen
I2C_WRITE	I ² C Befehl senden

9. Overlay Commands

Overlay erlaubt zusätzliche Grafikebenen.

Command	Beschreibung
OV	Overlay anzeigen
OV_ADD	Overlay erweitern

Paramtertabellen

Flags

Flag	Bedeutung
MASK_DRAW_BACKGROUND	Hintergrund zeichnen
MASK_DRAW_FOREGROUND	Vordergund zeichnen
WITHOUTLOAD	Patternbuffer nicht neu laden
WITHLOAD	Patterbuffer mit laden

SD Graphic System

API Reference der freien SGC Klasse

Die API erlaubt es, den Grafikcontroller über einfache Befehle vom Host aus zu steuern. Der Grafikcontroller übernimmt dabei vollständig die Darstellung auf dem OLED.

Der Host sendet nur **kompakte Befehle**, während Rendering, Animation und Sprite-Logik auf dem Grafikcontroller ausgeführt werden.

1. Grundfunktionen der Klasse sgc

```
#include <SGC.h>
```

print()

Schreibt Text auf das Display.

```
void sgc.print(const char* text);  
void sgc.print(const __FlashStringHelper* text);
```

Beschreibung

Der Text wird an der aktuellen Cursorposition ausgegeben.

Die Flash-Version erlaubt es, Strings direkt aus dem Programmspeicher zu senden und spart RAM auf dem Host.

Beispiel

```
sgc.print("Hello World");  
sgc.print(F("Hello from Flash"));
```

printAt()

Text an einer festen Position ausgeben. Die neue Cursorposition wird automatisch gesetzt.

```
void printAt(uint8_t x, uint8_t y, const char* text);
```

Beispiel

```
sgc.printAt(2,1,"Temperature");  
sgc.printAt(2,1,(F("Temperature")));  
Neue Cursorposition = 13
```

sdPrint()

Text mit internen Font des Graphiccontrollers ausgeben. Dieser besteht immer aus den ersten 120 Zeichen der Fontpage 0 aus der SD Oled Suite. Dieser Font wird automatisch von der SD beim Start des Graphiccontrollers in seinen Flashspeicher geladen. Es werden nur Änderungen in den Flashspeicher übernommen um Schreibzyklen einzusparen. Wird mit der SD Oled Suite in den ersten 120 Zeichen eine Änderung vorgenommen ist diese ohne weitere Maßnahme sofort verfügbar.

Beispiel:

```
sdPrint(F(„Hello World“));
```

Besonderheit:

Um nationale Zeichen zu unterstützen können Fonttiles als {[NUM]} eingefügt werden. Im Demofont 8x8-Font Demo P liegt 'Ä' auf 96. Um dieses Ä anzusprechen gilt folgender Syntax:

```
sdPrint(F(„{96}nderung“); // Ausgabe = Änderung
```

Damit können beliebige Zeichen definiert und über sdPrint() angesprochen werden. Sie müssen lediglich in der SD Oled Suite in den ersten 120 Zeichen des Fonts in der Fontpage 0 definiert werden.

Tip:

Verwenden sie Words! Sind schneller und universeller!

setCursor()

Cursorposition setzen.

```
void setCursor(uint8_t x, uint8_t y);
```

Beispiel

```
sgc.setCursor(0,2);  
sgc.print("Ready at Position 0,2");
```

home()

Cursor zurück an den Anfang.

```
void home();
```

2. Screen Funktionen

showScreen()

Zeigt einen Screen aus der Asset-Pipeline.

```
void showScreen(uint8_t screenNo);
```

Beschreibung

Ein Screen ist ein kompletter Displayinhalt, der auf der SD-Karte gespeichert ist. Die Darstellung eines Screens ist **nicht an Tiles** gebunden sondern Pixelgenau.

Beispiel

```
sgc.showScreen(24);
```

Anmerkung: Mit guten SD Karten beträgt die komplette Anzeigezeit ~18-20 ms

showScreens()

Spielt eine Animationssequenz aus mehreren Screens.

```
void showScreens(uint8_t from, uint8_t to, uint8_t delay);
```

Beispiel:

```
sgc.showScreens(100,120,30);
```

Spielt eine Animationssequenz (from, to, delay). Delay in ms. Es werden über SD RAW Multiread bis **50 fps** erreicht! Geringe Unterschiede → SD Qualität. **SDHC ergab bei Messungen keinen Vorteil.** Geschwindigkeit hängt nur vom SD Kartencontroller ab. Nicht zu verwechseln mit dem Cardreader!

setScreenPage()

Screens/Words und Fonts sind in Pages organisiert. Eine Page = 256 Elemente

```
void sgc.setScreenPage(uint8_t screenPage);
```

Beispiel

```
sgc.setScreenPage(1);
```

Setzt den Offset für **ALLE** weiteren Screenbefehle um 256 Screens weiter! Ermöglicht so Mehrsprachigkeit.

Beispiel

```
sgc.showScreen(0);           // zeigt Screen 0 an (z.Bsp: deutsch)
sgc.setScreenPage(1);        // setzt den Offset 255 Screens weiter
sgc.showScreen(0);           // zeig jetzt den Screen 255 an (z.Bsp.englisch)
sgc.setScreenPage(2);        // setzt den Offset 512 Screens weiter
```

```
sgc.showScreen(0);          // zeigt jetzt den Screen 512 an (z.Bsp französisch)
```

drawScreen()

Zeichnet einen Screen mit optionalen Pattern Masken.

```
void drawScreen(uint8_t screenNo,  
                uint8_t drawMask,  
                uint8_t loadMode);
```

Beispiel

```
sgc.drawScreen(0, MASK_DRAW_BACKGROUND, WITHOUTLOAD);
```

Zeigt Screen 0 an. Benutzt dabei Pattern Clipping. → loadPattern(). Es können damit nur (einer oder mehrere) Teilbereiche der OLED Anzeige neu geschrieben werden.

DrawScreens()

Zeichnet Screens mit optionalen Pattern Masken.

```
void drawScreen(uint8_t fromscreenNo, uint8_t toScreenNo,  
                uint8_t drawMask,  
                uint8_t loadMode);
```

Beispiel

```
sgc.drawScreens(0,32, MASK_DRAW_BACKGROUND, WITHOUTLOAD);
```

Zeigt Screen 0 – 32 an. Benutzt dabei Pattern Clipping. → loadPattern(). Es können damit nur (einer oder mehrere) Teilbereiche der OLED Anzeige neu geschrieben werden. Damit können Animationen in Teilbereichen abgespielt werden ohne den Rest zu beeinflussen.

3. Font Funktionen

setTilePage()

Fontpage wählen.

```
void setTilePage(uint8_t page);  
void setTilePage(0); // verwendet die ersten 256 Tiles des Fontbereiches
```

SD Oled kennt im Prinzip **unendlich viele Fonts**, zusätzlich zum eingebauten U8x8 Font. Diese können in der SD OLED Suite **direkt gezeichnet/bearbeitet** werden.

SetWordPage()

Wordpage wählen.

```
void setWordPage(uint8_t page);
```

Beispiel:

```
0 Englisch  
1 Deutsch  
2 Französisch
```

Words sind frei definierbare Texte die in der SD OLED Suite definiert werden.

Am ehestens vergleichbar mit StringArrays der normalen Programmierung, nur dass sie auch beliebige **grafische Elemente** beinhalten oder sein können. Sie stellen eine sehr flexible und performante Art der Ausgabe dar, da der Host nur 1 Byte für die Adressierung des Words braucht! Ihr Einsatz ermöglicht im Gegensatz zum normalen print() eine wesentlich höhere **Performance, Flexibilität und Mehrsprachigkeit**.

sgc.sdWordV();

Words können vertikal ausgegeben werden. Da Words auch graphische Element beinhalten oder sein können, können diese bei Bedarf auch um 90 Grad rotiert werden damit die Grafik richtig untereinander dargestellt werden kann.

Beispiel:

```
sgc.sdWordV(120,V_ROT90);
```

sgc.sdWordH();

Words horizontal ausgeben.

Beispiel:

```
sgc.sdWordV(120);
```

Words verbrauchen auf dem Host keinen Speicher da sie über die SD OLED Suite in beliebiger Anzahl definiert werden können und auf der SD in der Assetpipeline gespeichert werden.

Text mit Proportionalschrift ausgeben.

```
void sdPrintp(const char* text);  
void sdPrintp(F(const char* text));
```

Beispiel:

```
sgc.setFontPage(2);           // proportionalfont anwählen  
sgc.sdPrintp(F("Do proportional writing")); // text ausgeben
```

Benötigt einen ProportionalFont um maximal effektiv zu sein. Im Demo enthalten auf FontPage(2)!

Kann aber auch auf normale Fonts angewendet werden. Dann immer noch ein Effekt

4. Grafikfunktionen

drawLine()

Linie zeichnen.

```
void drawLine(uint8_t x1,  
              uint8_t y1,  
              uint8_t x2,  
              uint8_t y2);
```

Zeichnet eine Linie

drawRectangle()

Rechteck zeichnen.

```
void drawRectangle(uint8_t x,  
                  uint8_t y,  
                  uint8_t w,  
                  uint8_t h);
```

drawFilledRectangle()

Gefülltes Rechteck.

```
void drawFilledRectangle(uint8_t x,  
                        uint8_t y,  
                        uint8_t w,  
                        uint8_t h);
```

drawCircle()

Kreis zeichnen.

```
void drawCircle(uint8_t x,  
                uint8_t y,  
                uint8_t radius);
```

5. Sprite Funktionen

Sprites sind bewegliche grafische Elemente auf der obersten Ebene.

setSprite()

Sprite konfigurieren.

```
void setSprite(uint8_t spriteFrom,  
               uint8_t spriteCount,  
               uint16_t delay);
```

Ein Sprite ist eine Animation die als oberste Ebene gezeichnet wird. Screen – Overlay – Sprite. Im Gegensatz zum Overlay kann eine Sprite animiert sein.

SD Oled hält die **ersten 120 Tiles** des Fonts auf der Fontpage(0) **immer** im EEPROM (Flash beim LGT!) Das heisst der Zugriff ist um ein vielfaches schneller als 'normale' Tiles. Der Grafikprozessor lädt diesen Bereich beim **Start** immer **automatisch**, wobei nur geänderte Tiles überschrieben werden die das Board automatisch erkennt!

SpriteFrom gibt an wo der Beginn sein soll, SpriteCount gibt an wieviele Tiles animiert werden sollen. Delay gibt die Pause in ms. Dazwischen an.

Beispiel:

```
sgc.setSprite(1, 1, 0, 4, 50);
```

erzeugt ein animiertes Tile auf der XYPosition 1,1 ab der Fontposition 0 mit vier Animationssequenzen mit einem Delay von 50.

Abfolge der Ausgabe:

0 –delay–1–delay–2–delay–3 –delay–2–delay–1–delay–0..... immer auf der Pos.XY

In der Version 1.00 ist nur 1 Sprite möglich!

SetSpritePosition()

Neue Position des Sprites setzen.


```
void setSpritePosition(uint8_t x, uint8_t y);
```

6. Patternbuffer

Der Patternbuffer enthält eine Tile-Belegungsmap des aktuellen Screens.

1 Bit entspricht einem Tile.

Funktionen

```
void loadPattern(uint8_t screen, uint8_t patternFunction)
```

```
sgc.loadPattern(100,PATTERN_GET)
```

lädt das Pattern auf der ScreenNr.100 und überträgt das Pattern **zum Host** in den Patternbuffer. Damit können Kollisionen direkt auf dem Host abgefragt werden oder Pattern dynamisch modifiziert werden bevor sie wieder zum Grafikcontroller transferiert werden.

```
sgc.loadPattern(100,PATTERN_NOGET);
```

lädt das Pattern **nur in den GrafikController** für Clipping → siehe DrawScreen

```
sgc.clearPattern()
```

löscht das geladene Pattern

```
sgc.sdCollision(uint8_t x,uint8_t y);
```

liefert true wenn das Tile auf dem Hostpattern belegt ist. False wenn frei.

```
bool sgc.isPositionOccupied(uint8_t x, uint8_t y)
```

Ist ein **Hostfunktion** die den lokalen Patternbuffer befragt

Der Patternbuffer kann verwendet werden für:

Collision Detection

Dirty Region Rendering/Region Clipping

Anwendungsbeispiel:

Sie haben auf der Screenposition 100 eine definierte Eingabe/Ausgabemaske und wollen diese leeren.

```
sgc.loadPattern(100,NO_GET); // nur in Controller laden
```

```
sgc.printAt(4,4,"1,2345")          // z.Bsp. Messwert oder Eingabe
sgc.drawScreen(101, MASK_DRAW_BACKGROUND,WITHOUTLOAD);
```

(Screen 101 ist ein leerer Screen)-> Alle Maskeninhalte bleiben stehen. Gelöscht werden nur die Teile des Screens 100 die nicht belegt waren. Also 1,2345. Es können auch extra Patternscreens erstellt werden um gezielt Inhalte in Teilbereichen zu manipulierten.

Beispiele in PatternDemo.ino

7. Display Einstellungen

setFlipmode()

Display drehen.

```
void setFlipmode(uint8_t mode);
```

setContrast()

Kontrast einstellen.

```
void setContrast(uint8_t value);
```

setPowerSave()

Display in Power Save Mode setzen.

```
void setPowerSave(uint8_t mode);
```

setInverseFont()

Invertiert die Ausgabe von Zeichen

```
void setInverserFont(uint8_t mode); // 0 = normal/1 = invers
```

8. IO Funktionen

Der Grafikcontroller kann auch **einfache** IO-Aufgaben übernehmen.

setPin()

Digitalpin setzen.

```
void setPin(uint8_t pin, uint8_t value);
```

funktioniert nur mit den freien Pins des Graphikcontrollers. (D5,D6,D7)

setPinMode()

Pinmodus setzen.

```
void setPinMode(uint8_t pin, uint8_t mode);
```

funktioniert nur mit den freien Pins. (D5,D6,D7)

analogWrite()

```
void analogWrite(uint8_t pin, uint8_t value);
```

funktioniert an den Pins (A1-A7)des Graphikcontrollers

getKey()*

Tasteneingabe lesen.

```
uint8_t getKey();
```

***Nur am SD OLED PCB verfügbar. Siehe seperate Anleitung**

9. Overlays

Overlays erlauben zusätzliche grafische Ebenen.

Funktionen

```
overlayBegin()  
addOverlay(x,y,numOfTiles,Flags,len,time)
```

```
replaceOverlay(0,x,y,numOfTiles,Flags,len,time)
```

9. Cursorfunktionen

```
sgc.setCursor(uint8_t x,uint_8_t);
```

Beispiel:

```
sgc.setCursor(1,1);
```

Alle xy Koordinaten werden komprimiert übertragen. X,Y werden zu einem Byte!

10. Typischer Workflow

1. Screen/Word/Font im Editor erstellen
2. Animation, Fonts und Words erzeugen
3. Assets auf SD exportieren
4. SD in Grafikcontroller einsetzen
5. Host sendet Befehle

Beispiel:

```
sgc.showScreen(0);  
sgc.print("Hello");  
sgc.setCursor(0,2);  
sgc.print("Temperature:");
```